Energy Allocation Between Computation and Wi-Fi Transmission in TinyML Federated Learning: A Measured Crossover

Md Abdus Shakur Siam* and Farag Azzedin† *Department of Electrical and Electronic Engineering, Bangladesh University of Engineering and Technology (BUET), Dhaka, Bangladesh
shakursiam@gmail.com †Department of Information and Computer Science, King Fahd University of Petroleum and Minerals (KFUPM), Dhahran, Saudi Arabia
fazzedin@kfupm.edu.sa

***Abstract*—Wireless federated learning (FL) is usually designed on the assumption that radio transmission costs far more energy than on-device computation. We test this assumption on a microcontroller. An ESP32-S3 trains the last layer of a frozen keyword-spotting network and exchanges $3{,}120$-byte updates over 2.4 GHz Wi-Fi, while an independent INA238 monitor samples its supply at 1 kHz and GPIO markers separate computation from communication. To our knowledge, this is the first instrumented measurement of the computation–communication energy crossover for FL over Wi-Fi. In a 60-run campaign over $E \in \{1, 2, 5, 8, 10, 20\}$ local epochs, computation costs 1.00 mJ per epoch with no detectable constant term, and communication costs 9.7 mJ per client turn with no trend in E, so the two are equal at $E^\star \approx 9.7$. A second session gives 7.4 mJ and $E^\star \approx 7.4$, so the crossover depends on radio conditions. A 45-run test shows that training started immediately after a radio burst absorbs about 0.5 mJ of radio tail energy, which a 60 ms pause removes. Shannon-capacity and switching-energy models underestimate the energies by 95% to 99%, and a datasheet-current model is within about 10% only at its nominal point. We prove that sleeping between rounds moves the crossover toward the ratio of exchange time to epoch time; with assumed sleep powers this gives 11 to 19 epochs. We give a measurement-calibrated procedure for choosing E and design guidelines, and we state which quantities are measured and which are assumed.**

***Index Terms*—Federated learning, TinyML, energy efficiency, computation and communication trade-off, Wi-Fi, duty cycling, power measurement.**

I. Introduction

FEDERATED learning (FL) lets wireless devices train a shared model without sending their raw data [1]. Because the uplink is the scarce resource on smartphones, the usual advice is to run more local epochs E between rounds, which reduces the number of rounds and the traffic. The idea is older than FL: for wireless sensor nodes, sending one kilobit over 100 m was estimated to cost as much energy as three million instructions [2]. Much of the wireless FL literature follows this advice. Some works choose the number of local updates under a resource budget [3], [4], others allocate bandwidth, transmit power and local computation under analytical models [5]–[15], and others compress or quantize the updates [16]–[20] or study energy use in IoT settings [21], [22]. These models usually describe transmission through Shannon capacity and computation through a switching energy proportional to $\kappa f^2 C$ [5]–[7], [15]. In one simulated setting, Tran et al. find that computation energy is much smaller than communication energy, so that devices should compute more and communicate less [5].

Microcontrollers (MCUs) change this picture. An MCU trains on the same battery that feeds its radio, it has no accelerator, and its SRAM is too small to train a full network [23]. It also draws considerable power merely by staying awake, a term that the common models do not contain. The empirical literature is divided: some studies find that training dominates the energy budget [24], [25], whereas others, using low-rate radios, find that transfers dominate [26]–[28]. Because these studies differ in radio, model and energy accounting, they cannot locate the boundary on a single platform, and to our knowledge none of them locates it with an independent power instrument on a Wi-Fi FL loop.

This paper takes a wireless system design view. A device with a fixed energy budget can spend it on computation (more local epochs) or on transmission (more rounds). We ask three questions. At what E does the energy of computation in one client turn equal that of communication? How does this point move when the device sleeps between rounds or when the channel is busy? What does it imply for the rounds and airtime that an FL job uses? We answer the first question by measurement on one ESP32-S3 device, which plays the role of every client. This choice is deliberate: it isolates the physical energy of one client turn from differences between boards, and it is the setting in which the energy terms can be measured precisely. The answers to the second and third questions combine these measurements with a model that we state explicitly.

Our contributions are as follows.

1) We measure the energy of computation and communication per client turn on an ESP32-S3 over Wi-Fi with an independent power monitor, a fixed protocol and a phase-resolved attribution procedure (Algorithm 1), in a randomized main campaign with six values of E and ten repetitions each. Computation grows linearly with E and communication is approximately constant, so that the crossover lies near 9.7 epochs in the main campaign and near 7.4 in a second session.

Corresponding author: Farag Azzedin.

2) We isolate the post-communication radio tail in a 45-run test, quantify how much of it a training burst absorbs, and show that a short pause removes it.
3) We derive a closed-form crossover that includes sleep power and a contention factor, and we prove that sleep moves the crossover monotonically toward the ratio of exchange time to epoch time (Proposition 1).
4) We show how five common analytical models miss the measured energies and the crossover, and we explain why.
5) We break a client turn into protocol phases, link convergence to rounds, bytes and airtime, and give a measurement-calibrated procedure for choosing E (Algorithm 2). We separate the hardware crossover $E^\star$ from the learning optimum E_{opt} and state the status of every number.

Section II reviews related work, Section III gives the system model, and Section IV introduces the analytical baselines. Sections V to VIII present the testbed, the measurements, the sleep and contention analysis and the convergence study. Sections IX and X give design guidelines and limitations. Instrumentation details are in Appendix A, and the status of each result is listed in Appendix B.

TABLE I
KEY NOTATION.

Symbol	Meaning
E	Number of local epochs per client turn
K, D_k, B	Number of clients, local clips of client k, batch size
$\Delta E_{\mathrm{comp}}(E)$	Computation energy above baseline in one turn
ΔE_{comm}	Communication energy above baseline in one turn (download and upload)
α, c_0	Slope (mJ per epoch) and intercept of $\Delta E_{\mathrm{comp}}(E)$
P_{base}	Awake baseline power of the board
P_s, E_{wake}	Sleep power and wake-up energy
t_{ep}	Duration of one epoch
T_{comm}	Duration of the download and upload exchanges in one turn
$T_{\mathrm{active}}(E)$	Awake time of one turn, $T_{\mathrm{comm}} + t_{\mathrm{ep}}E$
T_r	Round period
$E_{\mathrm{turn}}(E, T_r)$	Gross energy of one client turn
$E^\star$, $E_0^\star$	Crossover; its value in standby
ρ	Time ratio $T_{\mathrm{comm}}/t_{\mathrm{ep}}$
$R(E)$, E_{opt}	Rounds to the target accuracy; energy-optimal E
κ_c	Airtime factor for channel contention
κ, f, C	Switching coefficient, CPU frequency and cycles (baseline model)
P_{tx}, M, SNR	Transmit power, payload and SNR (baseline model)

II. RELATED WORK

A. Analytical Energy Models for Wireless FL

Most energy-aware FL designs optimize an analytical model. Tran et al. trade learning time against device energy with a Pareto model [5]. Yang et al. minimize the total energy of FL under a latency constraint by choosing CPU frequencies and transmit powers [6], and Chen et al. choose users, resource blocks and transmit powers to reduce the FL loss under delay and energy limits [7]. Later work extends these models to IoT, cell-free, UAV and mobile edge networks [9]–[11], [13], [14]. The same structure remains in current designs: a recent personalized FL scheme for transformers computes its client energy from a computation term proportional to the capacitance coefficient and the square of the CPU frequency, and from an upload term whose rate follows the Shannon capacity of an OFDMA subchannel [15]. These models describe computation as $\kappa f^2 C$ and communication through Shannon capacity. They have no term for the power that a device draws while it is awake, and they are not checked against a measured device. Wang et al. and Luo et al. treat the number of local updates as a control variable under a resource budget [3], [4]. Both built prototypes, with three Raspberry Pi devices and two laptops over Wi-Fi [3] and with twenty Raspberry Pi devices [4]. Luo et al. state that they did not capture energy on their prototype because it is difficult to measure, and they draw it from a normal distribution instead [4].

B. Training and FL on Microcontrollers

Memory limits training on an MCU. Lin et al. train convolutional networks within 256 KB of SRAM by skipping the gradients of less important layers [23]. A simpler route freezes the feature extractor and trains only the last layer: TinyOL does this online on an Arduino Nano 33 BLE Sense [29], and TinyFedTL uses it for federated transfer learning [30]. Llisterri Giménez et al. run federated learning for keyword spotting on three Arduino Nano 33 BLE Sense boards [31]. Both federated systems exchange weights with the server over a serial port, and TinyFedTL reports more than 30 s for one transfer [30]. Neither system reports energy, and Llisterri Giménez et al. name the energy and bandwidth trade-offs of the network interfaces of an embedded device as future work [31]. Ficco et al. combine on-board transfer learning with FL over MQTT on ESP8266, ESP32 and Arduino nodes and report the training time and the current drawn during training, measured with a USB digital multimeter [24]; they do not measure the energy of communication. Other studies run FL on MCUs over LoRa [26]–[28]. Our model split follows the last-layer approach. Its backbone is the depthwise separable CNN for keyword spotting on MCUs [32], which is also the keyword-spotting model of the MLPerf Tiny benchmark [33].

C. Measured Energy of Radios and FL

Measurements often reveal costs that models leave out. Balasubramanian et al. measured 3G, GSM and Wi-Fi on mobile phones and found that 3G and GSM remain in high-power states after a transfer; in 3G, nearly 60% of the energy of a typical transfer is this tail energy [34]. For FL, Qiu et al. measured the power of NVIDIA Jetson clients with the built-in `tegrastats` utility but estimated communication energy from country-level download and upload speeds and the power of the router [35]. On MCUs, MLPerf Tiny measures inference energy with an external energy monitor [33], but it does not benchmark training or communication. Our work measures computation and Wi-Fi communication on one MCU with one independent instrument, includes the awake baseline and the radio tail, and compares the result with the analytical models above.

III. System Model

A. Protocol and Energy Accounting

A client holds D_k local audio clips. In each turn it fetches a configuration, downloads the global model head (3,120 B), runs E epochs of mini-batch SGD with batch size $B = 16$, and uploads the updated head (3,120 B). The host aggregates the uploads with FedAvg. Table I lists the notation.

We use two kinds of energy accounting, because the choice changes the answer. *Marginal energy* is the energy above the awake baseline power P_{base} and shows what computation and communication add. For one turn,

$$\Delta E_{\text{active}}(E) = \Delta E_{\text{comp}}(E) + \Delta E_{\text{comm}}, \tag{1}$$

$$\Delta E_{\text{comp}}(E) = \alpha E + c_0, \tag{2}$$

where ΔE_{comm} does not depend on E and covers the model download and upload. The configuration request is not part of it; Section VI-B shows what changes if it is counted. *Gross awake energy* also counts P_{base} over the active time

$$T_{\text{active}}(E) = T_{\text{comm}} + t_{\text{ep}} E, \tag{3}$$

where t_{ep} is the duration of one epoch. If the device sleeps for the rest of a period T_r at power P_s and pays a wake-up energy E_{wake}, the energy of one turn is

$$E_{\text{turn}}(E, T_r) = P_{\text{base}} T_{\text{active}}(E) + \Delta E_{\text{active}}(E) + P_s\big(T_r - T_{\text{active}}(E)\big) + E_{\text{wake}}. \tag{4}$$

Our main metric is the energy of one client turn; a round with K clients costs $K\,E_{\text{turn}}$.

B. Crossover and Design Problem

The crossover $E^\star$ is the number of epochs at which computation and communication cost the same. In standby, the condition is $\Delta E_{\text{comp}}(E^\star) = \Delta E_{\text{comm}}$, which gives $E_0^\star = (\Delta E_{\text{comm}} - c_0)/\alpha$. When the device sleeps between turns, the awake baseline during the active window adds to both terms, and the crossover becomes

$$E^\star(P_s) = \frac{(P_{\text{base}} - P_s)\, T_{\text{comm}} + \Delta E_{\text{comm}} - c_0}{(P_{\text{base}} - P_s)\, t_{\text{ep}} + \alpha}. \tag{5}$$

The following result describes how sleep moves the crossover.

Proposition 1: Let $\alpha > 0$, $t_{\text{ep}} > 0$ and $0 \le P_s \le P_{\text{base}}$, and define $\rho = T_{\text{comm}}/t_{\text{ep}}$ and $w = (P_{\text{base}} - P_s)t_{\text{ep}}/\big((P_{\text{base}} - P_s)t_{\text{ep}} + \alpha\big)$. Then

1) $E^\star(P_s) = (1 - w)\, E_0^\star + w\,\rho$, with $0 \le w < 1$;
2) $E^\star(P_s)$ lies between $E_0^\star$ and ρ;
3) as P_s decreases, $E^\star(P_s)$ moves monotonically toward ρ. It increases if $T_{\text{comm}} > E_0^\star\, t_{\text{ep}}$, decreases if $T_{\text{comm}} < E_0^\star\, t_{\text{ep}}$, and is constant if the two are equal.

Proof: Let $\Delta = P_{\text{base}} - P_s \ge 0$. Since $\Delta T_{\text{comm}} = \Delta t_{\text{ep}}\,\rho$ and $\Delta E_{\text{comm}} - c_0 = \alpha\, E_0^\star$, the numerator of (5) equals $\Delta t_{\text{ep}}\,\rho + \alpha\, E_0^\star$. Dividing by the denominator $\Delta t_{\text{ep}} + \alpha > 0$ gives part 1, and $0 \le w < 1$ because $\alpha > 0$. Part 2 follows because a convex combination lies between its two points. For part 3,

$$\frac{\partial E^\star}{\partial \Delta} = \frac{\alpha T_{\text{comm}} - t_{\text{ep}}(\Delta E_{\text{comm}} - c_0)}{(\Delta t_{\text{ep}} + \alpha)^2} = \frac{\alpha\, t_{\text{ep}}\,(\rho - E_0^\star)}{(\Delta t_{\text{ep}} + \alpha)^2}, \tag{6}$$

whose sign is that of $\rho - E_0^\star$, i.e., of $T_{\text{comm}} - E_0^\star t_{\text{ep}}$. Since Δ increases as P_s decreases, the claim follows. ∎

Proposition 1 explains the sleep results of Section VII. The weight w is the share of the awake baseline in the gross energy of one epoch. On our board, $P_{\text{base}} t_{\text{ep}}$ is about four times α, so w lies between 0.78 and 0.81 for every sleep state we consider, and the crossover moves about 80% of the way from $E_0^\star$ to ρ. This is why the choice of sleep state matters little and the exchange time matters a great deal.

The design problem is to choose E so that the energy to reach a target accuracy is smallest:

$$E_{\text{opt}} = \arg\min_E \; R(E)\, E_{\text{turn}}(E, T_r), \tag{7}$$

where $R(E)$ is the number of rounds needed. The crossover $E^\star$ is a property of the hardware and the protocol, whereas E_{opt} also depends on the data, the learning rate and the target. Because the two are different quantities, we do not use one to confirm the other.

C. Contention

When other devices use the channel, each exchange takes longer. We model this with an airtime factor $\kappa_c \ge 1$ on the communication energy, $\Delta E_{\text{comm}}(\kappa_c) = \kappa_c\, \Delta E_{\text{comm}}$. This is a modeling device; we did not measure κ_c.

IV. Analytical Baselines

We test five models that are common in the wireless FL literature, with parameters taken from the ranges in the cited works.

- **Shannon capacity.** The communication energy is $P_{\text{tx}} \cdot 8M/\big(B \log_2(1 + \text{SNR})\big)$, with B from 20 to 40 MHz, SNR from 10 to 30 dB [5], [6], [9], [10], P_{tx} from 100 to 200 mW [36], and $M = 2 \times 3120$ B.
- **Switching energy.** The compute energy per epoch is $\kappa f^2 C$ [5], [6], [8], [11], [12], [15], with $f = 240$ MHz and κ from 10^{-28} to 10^{-27}.
- **Combined.** The crossover is the Shannon communication energy divided by the switching compute energy.
- **Datasheet currents.** The datasheet transmit, receive and CPU currents [36] are multiplied by assumed durations at 3.3 V.
- **Communication dominant.** Communication always costs much more than computation, so that $E^\star \gg 100$ [1], [2].

Table II compares them with the main campaign of Section VI. Errors are relative to the measured $\Delta E_{\text{comm}} = 9.75$ mJ, $\alpha = 1.002$ mJ per epoch and $E^\star = 9.74$.

The models fail in different ways (Fig. 1). The first three predict energies that are 95% to 99% too low and a crossover with a very wide range. The communication-dominant premise places the crossover about 51 times too far out, so a designer who trusted it would choose the wrong operating point. The datasheet model lands within 10% at its nominal point, but we do not read this as evidence that the model is right: it assumes the burst durations, its range for $E^\star$ spans a factor of four, and it has no term for the 561 mW that the board

TABLE II
HOW ANALYTICAL MODELS MISS THE MEASURED ENERGIES AND CROSSOVER. NOMINAL VALUES USE THE PARAMETERS NAMED IN THE TEXT. RANGES SWEEP THE STATED BOUNDS. ERRORS ARE RELATIVE TO THE MAIN CAMPAIGN.

Model	ΔE_{comm} (mJ/turn)	ΔE_{comp} (mJ/epoch)	$E^\star$ (epochs)	Error	Direction	What the model leaves out
Shannon capacity	0.056 [0.013, 0.144]	n/a	n/a	-99.4% (comm.)	Too low	MAC and CSMA overhead, headers, HTTP exchange, analog radio draw
Switching energy	n/a	0.053 [0.011, 0.106]	n/a	-94.7% (comp.)	Too low	Memory stalls, SRAM and flash access, leakage
Combined	0.056	0.053	1.07 [0.12, 13.6]	-89.0% ($E^\star$)	Too low, very wide range	Both of the above
Datasheet currents	10.73 [6.75, 16.25]	1.012 [0.77, 1.23]	10.6 [5.5, 21.2]	$+10.1\%$, $+1.0\%$, $+8.8\%$	Within about 10% at the nominal point, wide range	Baseline draw, regulator loss, assumed burst times
Communication dominant	> 100	< 0.5	$\gg 100$ (nominal 500)	about $+5000\%$ ($E^\star$)	Far too high	Ignores that compute reaches parity near 10 epochs
Measured (this work)	**9.75**	**1.002**	**9.74** (A), **10.24** (B)	n/a	n/a	INA238, 5 V rail, $N = 60$

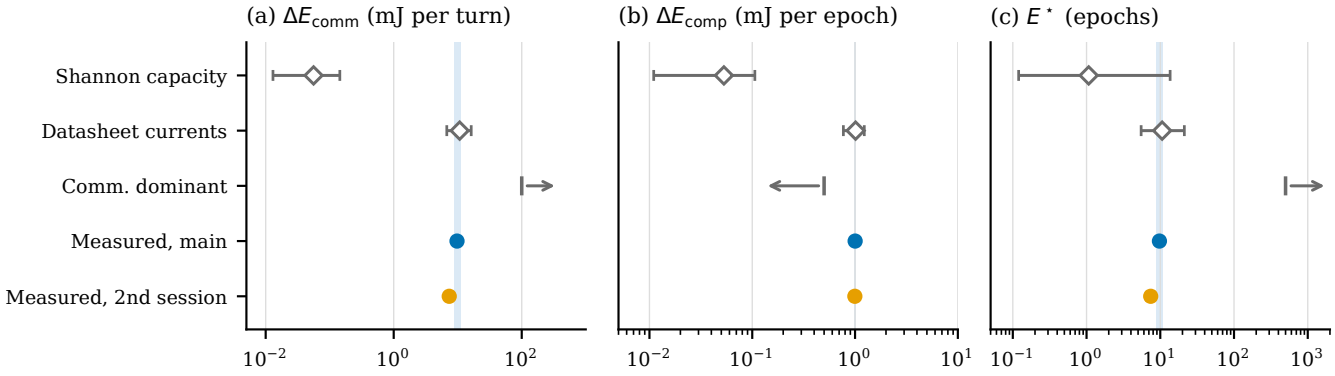


Fig. 1. Analytical models against the measurement, on log axes. Open diamonds are model nominal values with the range of Table II. Bars with arrows mark the one-sided bounds of the communication-dominant premise. Filled circles are measured: main campaign (blue, 95% bootstrap interval shaded) and second session (amber).

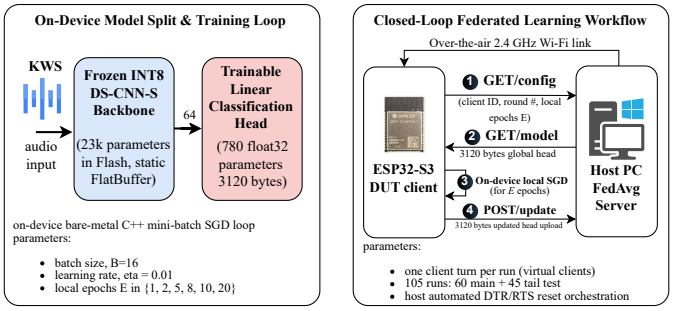


Fig. 2. Testbed. The ESP32-S3 device under test holds a frozen INT8 backbone and a trainable float32 head. An independent INA238 logger measures its supply power. A host runs FedAvg.

draws while awake, which is the largest energy in the system. Against the second session (Section VI-B), its communication energy would be 46% too high.

V. TESTBED AND METHOD

A. Model Split

The task is 12-class keyword spotting on Google Speech Commands v2 [37]. A frozen INT8 DS-CNN-S backbone [32], [33] with 23,552 parameters produces a 64-dimensional embedding. Because training the whole network would not fit in the 512 KiB of SRAM [23], the device trains only the float32 linear head, as in [29], [30]. The head has $64 \times 12 + 12 = 780$ parameters (3,120 B), runs SGD in bare-metal C++, and is the only part sent over the air. The K federated clients in our experiments are virtual: one physical device plays all of them. Each virtual client holds 100 clips, the learning rate is 0.01, and the batch size is 16.

B. Measurement

The device under test is an ESP32-S3 with two Xtensa LX7 cores at 240 MHz and an 802.11 b/g/n radio. A TI INA238 with a 15 mΩ (1%) high-side shunt sits in the supply line, and a second ESP32-S3 reads it at 1 kHz and streams the samples at 921,600 baud. The device raises GPIO markers during training and during HTTP exchanges, and a 3-bit phase bus labels sub-phases, so that every sample is tagged as training, communication or idle. Energy is the time integral of power. The board is supplied with 5 V DC at its input, which feeds the on-board 3.3 V regulator. Appendix A gives the wiring and isolation steps, and Fig. 2 summarizes the testbed.

All 105 runs use one firmware build, one board pair, one access point and fixed cables. The access point is a phone

Algorithm 1 Phase-resolved energy attribution and run acceptance

Require: Trace $\{(t_n, P_n, g_n^{\mathrm{tr}}, g_n^{\mathrm{co}}, d_n)\}$ at $\Delta t = 1\,\mathrm{ms}$: power, training and communication marker lines, dropped-sample count; expected training time $\hat{T}(E)$

$\mathcal{T}_1, \ldots, \mathcal{T}_J \leftarrow$ pulses of g^{tr}; $\mathcal{C}_1, \ldots, \mathcal{C}_S \leftarrow$ pulses of g^{co}, in time order
if $\sum_n d_n > 0$ **or** t not strictly increasing **or** $J \neq 1$ **or** $S < 3$ **or** $|\mathcal{T}_1|\Delta t \notin [0.6, 1.5]\,\hat{T}(E)$ **then**
  return REJECT
end if
$n_a \leftarrow$ first marker sample; $n_b \leftarrow$ last marker sample
$b_1 \leftarrow \mathrm{median}\{P_n : 5\,\mathrm{s} \le t_n - t_0 < 25\,\mathrm{s}\}$
$b_2 \leftarrow \mathrm{median}\{P_n : 5\,\mathrm{s} \le t_n - t_{n_b} < 25\,\mathrm{s}\}$
if $t_{n_a} - t_0 < 25\,\mathrm{s}$ **or** $|b_2 - b_1|/b_1 > 2\%$ **then**
  return REJECT
end if
$P_{\mathrm{base}} \leftarrow (b_1 + b_2)/2$
$\Delta E_{\mathrm{comp}} \leftarrow \sum_{n \in \mathcal{T}_1} (P_n - P_{\mathrm{base}})\Delta t$ ▷ negative terms kept
$\Delta E_{\mathrm{cfg}}, \Delta E_{\mathrm{dl}}, \Delta E_{\mathrm{ul}} \leftarrow \sum_{n \in \mathcal{C}_s} (P_n - P_{\mathrm{base}})\Delta t$ for $s = 1, 2, 3$
$\Delta E_{\mathrm{comm}} \leftarrow \Delta E_{\mathrm{dl}} + \Delta E_{\mathrm{ul}}$; $T_{\mathrm{comm}} \leftarrow (|\mathcal{C}_2| + |\mathcal{C}_3|)\Delta t$
return ACCEPT and P_{base}, ΔE_{comp}, ΔE_{cfg}, ΔE_{comm}, T_{comm}

hotspot on channel 6 at 0.5 m from the board, and the signal strength was −40 to −41 dBm at the start and end of every run. Each run has 30 s of idle before the turn and 30 s after it. Algorithm 1 gives the energy attribution and the acceptance test that we apply to every trace. The baseline power of a run is the median power from 5 s to 25 s of each idle period, averaged over the two periods, and the energy above baseline is the sum of $(P - P_{\mathrm{base}})\,\Delta t$ over the samples of a phase, with negative terms kept. A run passes only if it has no dropped samples, strictly increasing timestamps, the expected number of markers and a baseline drift below 2% between the two idle periods. Failed attempts were kept and repeated; twenty attempts failed, mostly because the training marker never appeared. No sample was repaired or interpolated.

We ran two campaigns. The *main campaign* has 60 runs, with $E \in \{1, 2, 5, 8, 10, 20\}$, ten repetitions each, a randomized run order and a 60 ms pause between the phases of a turn. The *tail test* has 45 runs, with $E \in \{1, 5, 20\}$, pauses of 0, 60 and 200 ms and five repetitions per cell. It ran in five blocks, each of which went once through all nine cells in a fixed order.

C. Power Profile

Figure 3 shows one run of the main campaign. With modem sleep disabled, the awake idle board draws $P_{\mathrm{base}} = 561.1 \pm 1.8$ mW over the 105 runs, and the baseline of single runs ranges from 557.8 to 569.7 mW. In the run shown, the baseline is 560.5 mW and the SGD plateau is 691.5 mW, which is 131 mW above baseline, for 152 ms. Across the campaign, one epoch takes 7.58 ms and costs 1.00 mJ above baseline, i.e., 132 mW. Single samples in the radio phases reach about 1 W. The 60 ms pauses let the radio tail decay before training starts (Section VI-C).

TABLE III
MEASURED MARGINAL ENERGY PER CLIENT TURN IN THE MAIN CAMPAIGN ($N = 10$ PER REGIME). VALUES ARE THE MEAN ± THE STANDARD DEVIATION. THE RATIO IS THE MEAN COMPUTATION ENERGY OVER THE MEAN COMMUNICATION ENERGY.

E	T_{train} (ms)	ΔE_{comp} (mJ)	ΔE_{comm} (mJ)	Median comm.	Ratio
1	7.5 ± 0.5	0.99 ± 0.06	8.5 ± 3.2	7.5	0.12
2	15.1 ± 0.3	2.00 ± 0.06	9.4 ± 2.9	9.0	0.21
5	37.9 ± 0.3	5.01 ± 0.10	10.6 ± 6.9	8.5	0.47
8	60.7 ± 0.5	8.01 ± 0.19	11.3 ± 5.0	9.6	0.71
10	75.8 ± 0.4	9.98 ± 0.25	7.9 ± 2.2	7.5	1.27
20	151.6 ± 0.5	20.04 ± 0.37	10.8 ± 4.3	11.1	1.86

VI. Measured Crossover

A. Energy per Turn

Table III and Fig. 4 give the measured energy per client turn in the main campaign. Computation grows almost linearly with E and has a small spread. Communication varies from 8.6 to 11.9 mJ with no clear trend; a linear fit of communication energy on E has a slope of 0.08 ± 0.09 mJ per epoch.

A least-squares fit of the 60 runs gives

$$\Delta E_{\mathrm{comp}}(E) = 1.002\,E - 0.010\ \mathrm{mJ}, \qquad R^2 = 0.999, \quad (8)$$

with 95% bootstrap intervals of $[0.991, 1.014]$ for the slope and $[-0.070, +0.049]$ for the intercept, so the intercept is zero within about 0.07 mJ. The slope matches the product of the plateau power above baseline and the epoch time, $132\,\mathrm{mW} \times 7.58\,\mathrm{ms} = 1.00\,\mathrm{mJ}$. The mean communication energy over the 60 runs is $\Delta E_{\mathrm{comm}} = 9.75$ mJ, with a 95% interval of $[8.77, 10.96]$.

Communication energy scatters much more than computation energy: its standard deviation is about 2 to 7 mJ, against 0.06 to 0.4 mJ for computation. The scatter comes from the radio exchange. Most exchanges take 85 to 163 ms (the middle half of all 105 runs), but some stall (Fig. 5a). In 6 of the 105 runs the exchange took more than 500 ms, and the longest took 2.8 s. A stall adds energy because the radio stays on, which is why the mean (9.75 mJ) lies above the median (8.85 mJ). Duration alone does not explain the spread, however: across all runs, the correlation between exchange time and communication energy is only $r = 0.31$ (0.56 without the six stalls).

B. Crossover

The standby crossover is $E_0^\star = (\Delta E_{\mathrm{comm}} - c_0)/\alpha$. We state it in three ways.

- **Version A.** With the communication energy of the two model exchanges as set by the markers, $E^\star = 9.74$ epochs, with a 95% bootstrap interval of $[8.79, 10.85]$.
- **Version B.** Charging a 0.506 mJ radio tail, estimated in a pilot campaign, to communication gives $E^\star = 10.24$ epochs, with an interval of $[9.30, 11.36]$. Section VI-C measures the tail directly.

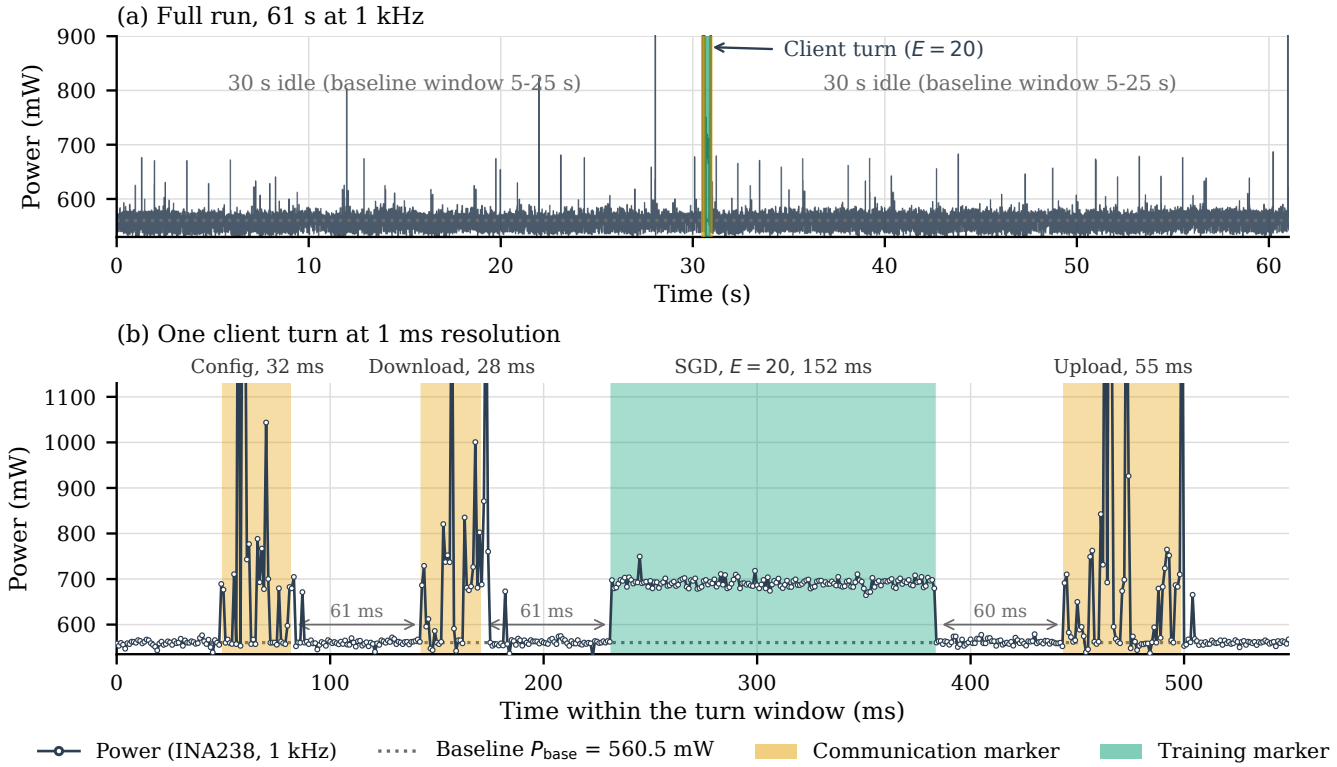


Fig. 3. Power trace of one main-campaign run ($E = 20$) at 1 kHz. (a) The full 61 s run, with 30 s of idle before and after the turn. (b) One client turn: configuration request, model download, SGD, model upload and the 60 ms pauses between them.

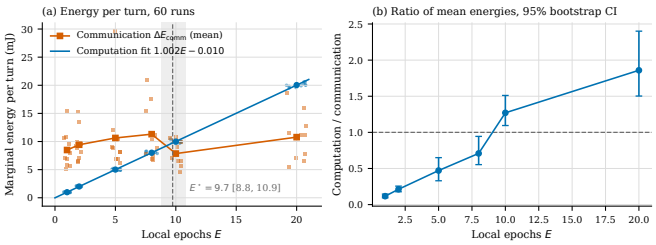


Fig. 4. Measured marginal energy per client turn in the main campaign. (a) Each point is one run. Computation grows with E. Communication is about flat but scattered. The curves meet near $E = 10$; the shaded band is the 95% bootstrap interval of $E^\star$. (b) The ratio of the mean energies crosses one at the same point. Bars are 95% bootstrap intervals; the dip between $E = 8$ and $E = 10$ lies within them.

- **Configuration included.** The configuration request costs 3.4 mJ on average. Counting it as communication gives $E^\star = 13.1$ epochs, with an interval of $[11.9, 14.4]$.

Below the crossover, communication takes most of the marginal energy, and above it computation does; at $E = 20$, computation is 65% of the marginal energy of a turn. Figure 6 collects these estimates together with those discussed next.

The main campaign is not the only measurement. The tail test has 15 runs at the same 60 ms setting with $E \in \{1, 5, 20\}$, which give $\Delta E_{\text{comm}} = 7.37$ mJ and $E^\star = 7.42$ epochs, with an interval of $[6.54, 8.36]$. For the same values of E, the main campaign gives 9.97 mJ, a difference of 2.6 mJ (Welch $t = 2.5$, permutation test $p = 0.05$). A pilot campaign with a different logger setup gave 7.33 mJ and $E^\star = 7.4$. The difference is not a fixed offset between sessions (Fig. 5b). In the main campaign, the mean communication energy of the first, second and last third of the runs is 11.8, 8.1 and 9.3 mJ (Spearman correlation with run order -0.23), whereas in the second session it is 8.2, 7.9 and 7.6 mJ. Because the main campaign was randomized, this drift is spread over all values of E and does not bias the slope. We did not log retries or the load of the phone hotspot, so we cannot attribute the change in radio cost within or between sessions to a cause. The variation between sessions is, however, larger than the tail effect of the next subsection. We therefore report the crossover as about 7.5 to 10 epochs, with the main campaign, which has the most runs, at the upper end. The computation side is stable: the tail test gives a slope of 0.996 mJ per epoch at 60 ms, in agreement with 1.002.

C. The Radio Tail

After a radio burst, the power takes a short time to return to the baseline. Cellular radios are known for long tails that can take most of the energy of a transfer [34]. The Wi-Fi tail here is much shorter, but if training starts at once, the end of the tail falls into the training window. The tail test checks this by comparing back-to-back phases (0 ms) with pauses of 60 ms and 200 ms.

Table IV and Fig. 7 show the result. With no pause, the intercept is $+0.48$ mJ, and with a 60 ms pause it is -0.01 mJ. The difference is 0.49 mJ, with a 95% interval of $[0.18, 0.82]$,

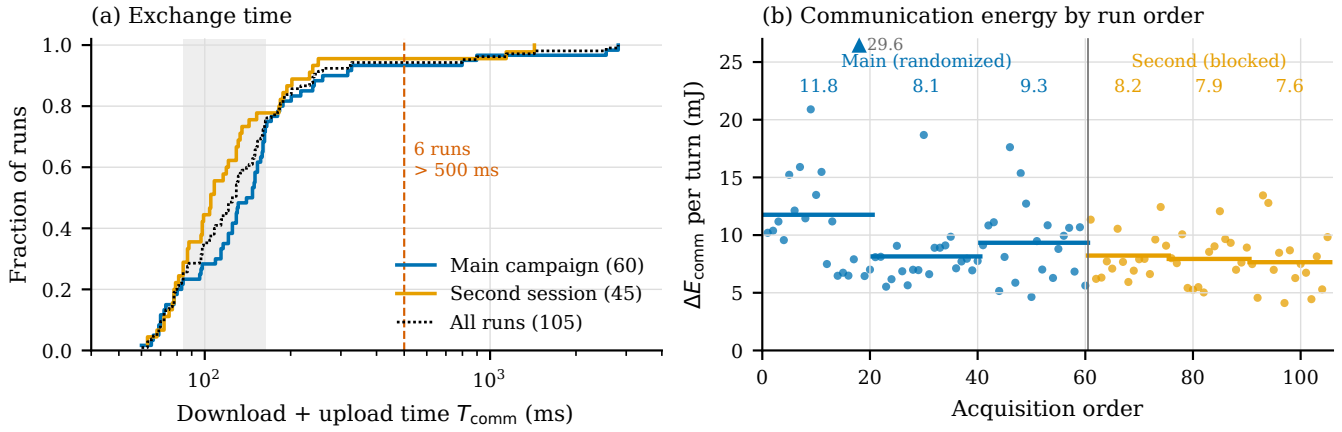


Fig. 5. Communication diagnostics for all 105 runs. (a) Cumulative distribution of the download and upload time; the shaded band is the middle half of the runs. (b) Communication energy per turn in acquisition order. Horizontal bars and numbers give the mean of each third of a campaign. One main-campaign run (29.6 mJ) is off scale and marked at the top edge.

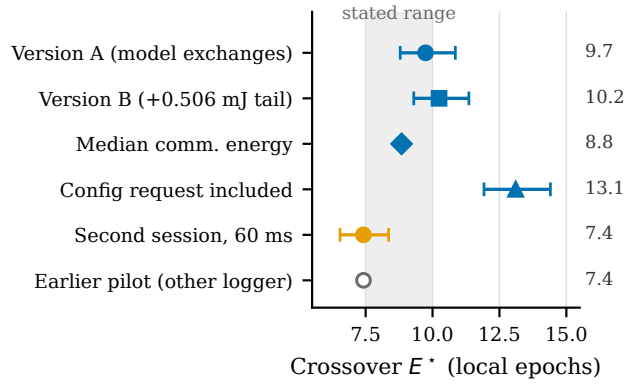


Fig. 6. Estimates of the marginal crossover $E^\star$ with 95% bootstrap intervals where available. Blue: main campaign under different accounting. Amber: second session at the same 60 ms setting. Open circle: pilot campaign with a different logger (not used elsewhere). The shaded band is the range of about 7.5 to 10 epochs stated in the text.

TABLE IV
LINEAR FIT OF TRAINING ENERGY AGAINST E FOR EACH SETTLING DELAY. EACH ROW HAS 15 RUNS, FIVE EACH AT $E = 1$, 5 AND 20. INTERVALS ARE 95% BOOTSTRAP INTERVALS.

Delay (ms)	Slope α (mJ/epoch)	Intercept c_0 (mJ)	R^2
0	1.020 [0.997, 1.048]	+0.475 [0.182, 0.786]	0.997
60	0.996 [0.973, 1.026]	−0.014 [−0.112, 0.075]	0.998
200	1.006 [0.993, 1.017]	+0.020 [−0.033, 0.077]	1.000

and a permutation test that shuffles the delay labels within each E gives $p = 0.04$. A pause of 200 ms gives the same result as 60 ms; their intercepts differ by 0.03 mJ, with an interval of $[-0.07, 0.15]$. The evidence is moderate, because each cell has only five runs and the extra energy at 0 ms differs across E: it is 0.66, 0.43 and 1.02 mJ at $E = 1$, 5 and 20.

The traces show where the extra energy comes from. Figure 8 shows the mean power above baseline in the 60 ms pause after each phase. The pauses after the model download,

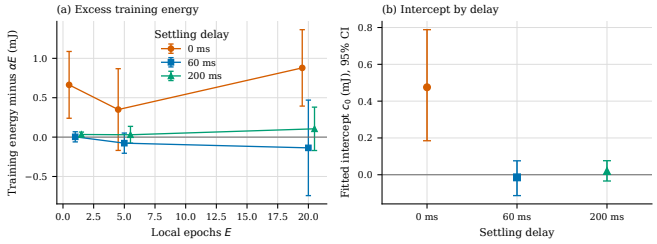


Fig. 7. Tail test. (a) Training energy above the line αE, with $\alpha = 1.002$, for three settling delays. Bars show one standard deviation. (b) The fitted intercept for each delay, with its 95% interval.

the configuration request and the upload hold 0.75, 0.76 and 0.59 mJ, most of it within 20 ms, whereas the pause after training holds only 0.11 mJ. A radio burst thus leaves about 0.6 to 0.8 mJ behind, and a training burst leaves little. In the five back-to-back runs at $E = 20$, the first 10 ms of training draw on average 171 mW above baseline, compared with 129 to 133 mW when training starts after a pause. These numbers agree with the extra 0.5 to 1 mJ in the fit.

Two consequences follow. First, a measurement that starts training immediately after a radio burst overstates the compute energy by about 0.5 mJ, which is half an epoch at $E = 1$. Second, the tail is a real communication cost that is paid after every burst. If both the download and the upload tails were charged to communication, its cost would rise by about 1.1 mJ and $E^\star$ by about 1.1 epochs; Version B, which charges a single tail of 0.506 mJ, is therefore a lower bound. The pause is a measurement tool: a deployed firmware need not wait for the tail, but the tail energy is paid either way.

D. Link-Layer Breakdown

The phase bus shows where the time of an exchange goes. Table V lists the result for all 105 runs. The payload itself is small: moving 3,120 B at an OFDM rate takes less than a millisecond. Even so, the download takes about 37 ms and the upload about 85 ms, so most of the time goes to the protocol

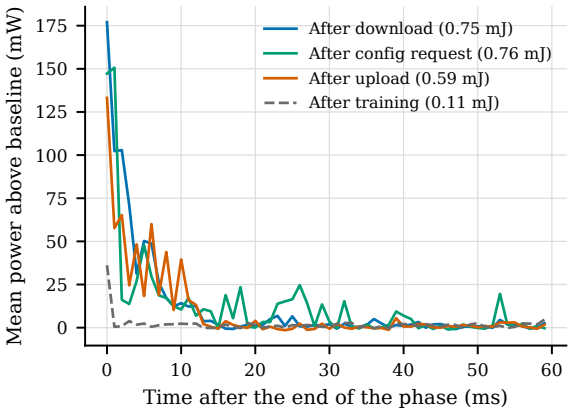


Fig. 8. Mean power above baseline after the end of each phase, taken from the 60 ms pauses of 74 runs. The numbers in the legend are the energy in the first 60 ms.

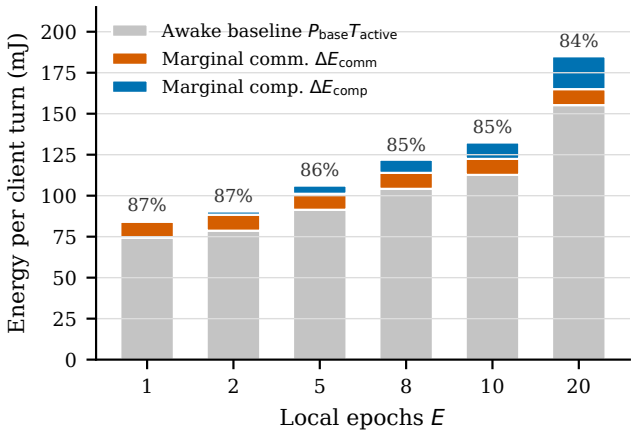


Fig. 9. Gross awake energy of one client turn in standby, from the measured fits with $T_{\text{comm}} = 125$ ms. The configuration request is not included. Numbers above the bars are the share of the awake baseline $P_{\text{base}}T_{\text{active}}$.

TABLE V
PHASES OF ONE CLIENT TURN FROM THE PHASE BUS. TIMES ARE THE MEDIAN WITH THE MIDDLE HALF OF THE RUNS IN BRACKETS, FOR ALL 105 RUNS. ENERGIES ARE THE MEAN ± THE STANDARD DEVIATION ABOVE BASELINE.

Phase	Protocol state	Time (ms)	Energy (mJ)
Config request	HTTP GET of the configuration	41 [28, 69]	3.3 ± 2.5
Download	HTTP GET of the model head	37 [27, 57]	3.6 ± 1.7
Training	Local SGD (E epochs)	$7.58E$	$1.00E$
Upload	HTTP POST of the update	85 [46, 115]	5.4 ± 3.0
Download and upload		125 [84, 163]	9.0 ± 3.7

TABLE VI
CROSSOVER FROM EQ. (5) AND ENERGY PER TURN AT $T_r = 60$ S, USING $T_{\text{comm}} = 125$ MS FOR THE ENERGIES. THE SLEEP POWER AND WAKE-UP ENERGY ARE ASSUMED VALUES.

State	P_s (mW)	E_{wake} (mJ)	$E^\star$ for $T_{\text{comm}} =$		E_{turn} (J)		
			85 ms	160 ms	E=1	E=5	E=20
Standby	561.1	0	9.7	9.7	33.68	33.68	33.70
Modem sleep	100	0	10.9	18.6	6.07	6.09	6.16
Light sleep	10	0.5	10.9	18.9	0.685	0.705	0.783
Deep sleep	0.08	180	10.9	18.9	0.270	0.291	0.370

and the radio rather than to the bits. This also explains why the Shannon model fails. The upload costs more than the download because the radio transmits, and the configuration request, which carries almost no data, costs as much as the model download. The communication time of a turn is 125 ms at the median, with a long tail from stalls. The sleep analysis below uses T_{comm} from 85 to 160 ms, the middle half of the runs.

VII. SLEEP AND CHANNEL CONTENTION

The sleep powers and wake-up energies in this section are datasheet-style assumptions that we did not measure on our board, and we did not measure channel contention. We use them to show how the problem changes, not to claim savings.

A. Sleep Moves the Crossover

In standby, the awake baseline is paid whether or not the device works, so it drops out of the marginal comparison. If the device sleeps between turns, the baseline during the active window becomes a real cost, and both a longer exchange and every additional epoch cost more. One epoch takes 7.58 ms at 561 mW, which is 4.3 mJ, more than four times its marginal 1.00 mJ. Figure 9 shows the size of this term: the awake baseline over the active window is 84% to 87% of the gross energy of a turn for every E we tested.

Evaluating Eq. (5) gives Table VI and Fig. 10a. The crossover rises from 9.7 to between 11 and 19 epochs as soon as sleep is possible, and the curves for modem, light and deep sleep almost coincide. Both observations follow from Proposition 1: with $w \approx 0.8$, the crossover moves most of the way from $E_0^\star = 9.74$ toward $\rho = T_{\text{comm}}/t_{\text{ep}}$, which is 11.2, 16.5 and 21.1 for $T_{\text{comm}} = 85$, 125 and 160 ms. The exchange time therefore matters much more than the sleep state, which is why the stalls of Section VI-D matter for a designer. The threshold of part 3 of the proposition is $E_0^\star t_{\text{ep}} = 73.9$ ms. Only 14 of the 105 measured exchanges were shorter, so for most turns sleep raises the crossover.

Two further points hold even with assumed values. In standby, a 60 s turn costs about 33.7 J regardless of E, so idling dominates. In deep sleep, the same turn costs about 0.3 J, and the wake-up energy and the active window dominate. The value of a saved round therefore depends on the sleep regime, which is a power-management choice rather than a learning hyperparameter.

B. Contention

With the airtime factor of Section III, the standby crossover is $E^\star(\kappa_c) = (\kappa_c \cdot 9.748 + 0.0098)/1.002$, which gives 9.7, 14.6, 19.5 and 29.2 for $\kappa_c = 1$, 1.5, 2 and 3 (Fig. 10b). If the device sleeps and contention also stretches the awake exchange time, the effect is stronger: with deep sleep and $T_{\text{comm}} = 125\kappa_c$ ms, the crossover grows from 15.2 at $\kappa_c = 1$ to 45.6 at $\kappa_c = 3$. The direction is clear: a busy channel lengthens each exchange and pushes the crossover up, so more local work per round

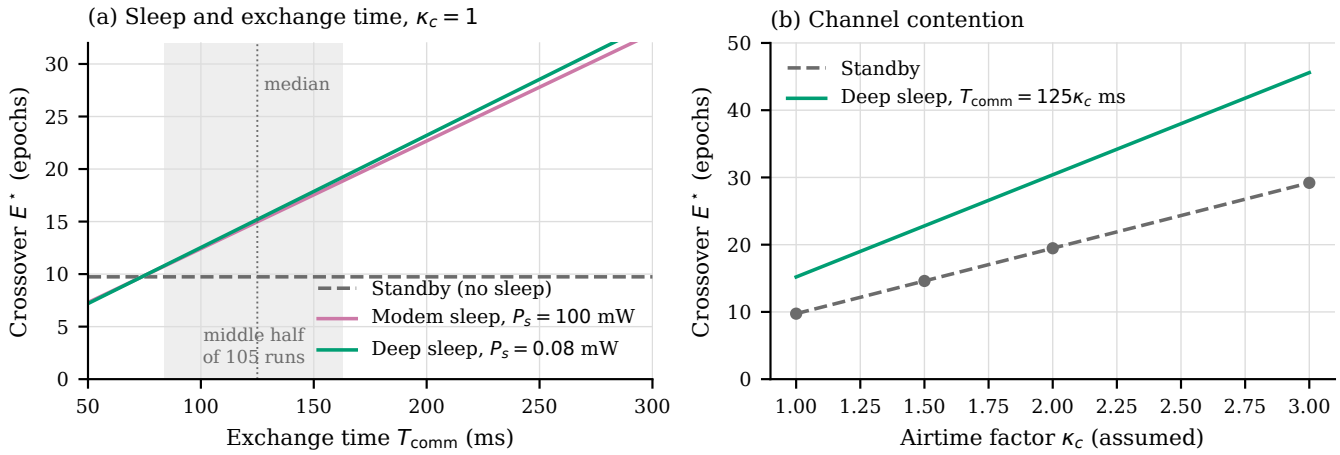


Fig. 10. Crossover from Eq. (5) with measured α, c_0, ΔE_{comm}, P_{base} and t_{ep}, and assumed sleep powers. (a) Against the exchange time; light sleep (not drawn) lies on the deep-sleep curve. The shaded band is the middle half of the measured exchange times. (b) Against the airtime factor κ_c. In standby only the communication energy scales with κ_c. For deep sleep, the exchange time is also scaled, $T_{\text{comm}} = 125\kappa_c$ ms. All sleep and contention values are model-only.

pays off. The size of the effect remains a hypothesis, because we did not vary the channel load.

VIII. Convergence, Rounds and Spectrum Use

A. Setup

On easy tasks every setting reaches the target in one round, which hides the trade-off, so we use a harder simulated task. The simulation trains the same 780-parameter head with FedAvg on embeddings from the frozen backbone of Section V, computed from clips mixed with background noise at 10 dB SNR. The training pool has 200 clips for each of the ten keywords and the unknown class; the silence class is not included. There are $K = 20$ clients with 100 disjoint clips each, Dirichlet label skew with concentration 0.3, batch size 16, up to 30 rounds and 20 random seeds. The 1,650 held-out clips are split per class into a validation half, used only to choose the learning rate, and a test half, used for every reported number. For each E, the learning rate is the value in $\{0.003, 0.01, 0.03, 0.1, 0.3, 1.0\}$ that minimizes the mean number of rounds to 75% validation accuracy; the selected rates are 0.3 for $E \leq 2$ and 1.0 otherwise. Rounds to 75% test accuracy are fractional and found by linear interpolation between rounds. All 20 seeds reach the target for every E, and the final test accuracy after 30 rounds is 78.7% to 79.4%.

B. Rounds, Bytes, Airtime and Energy

Table VII lists the simulated rounds $R(E)$ together with energies computed from them. We combine $R(E)$ with the fitted model of Eq. (2), the main-campaign ΔE_{comm} and $T_{\text{active}} = 125 + 7.58E$ ms, and, for the last column, the assumed deep-sleep values. One client moves $2 \times 3{,}120 = 6{,}240$ B per round, so traffic and channel accesses to the target scale with $R(E)$. From $E = 1$ to $E = 20$, the traffic per client falls from 34.1 kB to 8.0 kB, and the number of rounds, each with a download and an upload, falls from 5.5 to 1.3.

TABLE VII
Rounds and energy per client to reach 75% test accuracy in the $K = 20$ non-IID simulation (mean over 20 seeds, 95% bootstrap interval for $R(E)$). Marginal and gross energies use the measured fit. The last column assumes $P_s = 0.08$ mW, $E_{\text{wake}} = 180$ mJ and $T_r = 60$ s. Bold marks the minimum of each column.

E	η	$R(E)$	Traffic (kB)	Marginal (J)	Gross (J)	Deep sleep (J)
1	0.3	5.47 [4.93, 6.08]	34.1	0.059	0.465	1.48
2	0.3	3.18 [2.83, 3.54]	19.9	0.037	0.288	0.88
5	1.0	2.06 [1.82, 2.36]	12.9	0.030	0.219	0.60
8	1.0	1.61 [1.41, 1.81]	10.0	0.029	0.196	0.49
10	1.0	1.33 [1.19, 1.48]	8.3	**0.026**	**0.177**	**0.42**
20	1.0	1.28 [1.16, 1.40]	8.0	0.038	0.236	0.47

C. Crossover and Learning Optimum

Under all three accountings, the energy to the target is smallest at $E = 10$ (Table VII, Fig. 11). The optimum is not sharp. Across seeds, the energy-optimal E is 5, 8 or 10 in 18 of 20 seeds for marginal energy and 8 or 10 in 14 of 20 seeds for gross energy, and with deep sleep it is 20 in 4 seeds. The accounting changes the cost of moving away from the optimum. Relative to $E = 10$, $E = 5$ costs 15%, 24% and 42% more in marginal, gross and deep-sleep energy, whereas $E = 20$ costs 44%, 34% and 12% more. Sleep thus penalizes too little local work and tolerates too much, in line with the higher crossover of Section VII. The behaviour of $R(E)$ explains the optimum. Rounds fall quickly up to $E = 10$ and then reach a floor: $R(20)$ is only 4% below $R(10)$, while the compute energy per turn doubles. The floor is probably set by how well clients agree on skewed data; client drift may contribute, but we did not test it. That the learning optimum lies near the marginal crossover is a property of this task and is not evidence for either quantity. On an easier task, the same hardware would give a different E_{opt}. The crossover alone therefore does not decide the best setting; the radio, the sleep schedule and the data all play a part.

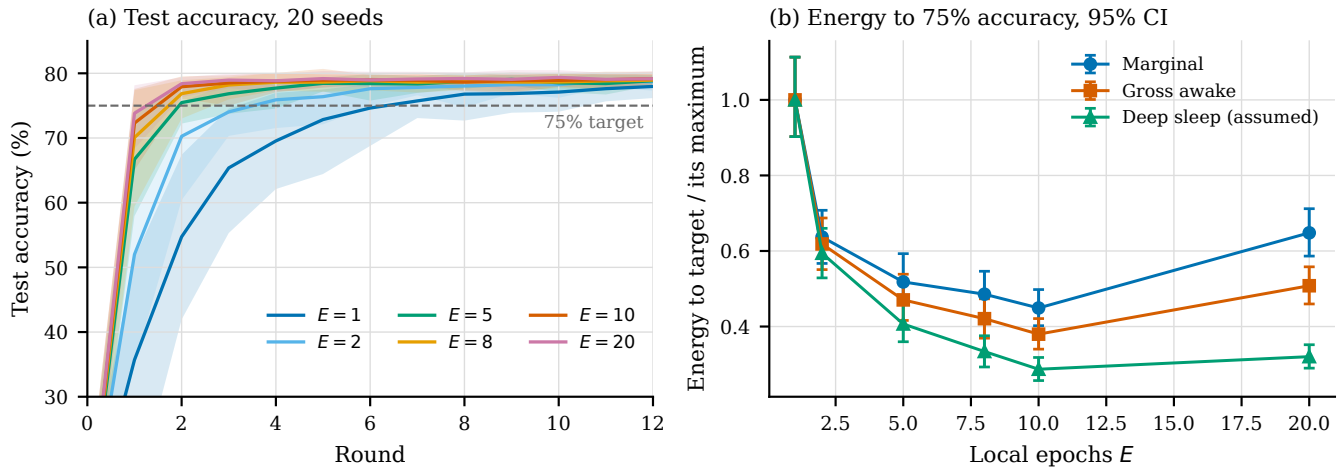


Fig. 11. Convergence in the $K = 20$ non-IID simulation. (a) Test accuracy per round, mean and 2.5 to 97.5 percentile range over 20 seeds. (b) Energy to 75% test accuracy for each accounting, divided by its own maximum, with 95% bootstrap intervals over seeds. The deep-sleep values are model-only.

Algorithm 2 Measurement-calibrated choice of local epochs

Require: Candidate set $\mathcal{E}$; sleep state (P_s, E_{wake}); round period T_r; target accuracy

1: Measure N turns per E for a few values of E with an external monitor and phase markers; fit α, c_0 by least squares; estimate P_{base}, t_{ep}, ΔE_{comm} and T_{comm}
2: Compute $E^\star(P_s)$ from Eq. (5) and $\rho = T_{\text{comm}}/t_{\text{ep}}$
3: **for** each $E \in \mathcal{E}$ **do**
4: Obtain $R(E)$ for the target accuracy
5: $J(E) \leftarrow R(E)\, E_{\text{turn}}(E, T_r)$ from Eq. (4)
6: **end for**
7: **return** $E^\star(P_s)$ and $E_{\text{opt}} = \arg\min_{E \in \mathcal{E}} J(E)$

D. Measurement-Calibrated Choice of E

Algorithm 2 turns the model into a procedure that a designer can apply to a new device or radio. Its first line needs only a short measurement campaign, such as the one in Section V, and line 4 needs $R(E)$ from a simulation or a pilot FL run of the target task. The procedure returns both the crossover and the energy-optimal E, so that the two are kept separate.

IX. Design Guidelines

1) **Do not assume that communication dominates on an MCU.** On this platform the marginal energies are equal between about 8 and 10 epochs. The common choice of $E = 5$ to 10 lies below or near that point, so communication remains the larger marginal term over most of that range, but computation is not negligible.
2) **Count the cost of being awake.** At $E = 1$, the awake baseline over a turn is about 74 mJ, whereas the marginal energy is about 11 mJ. Sleep scheduling can matter more than tuning E.
3) **Treat the round period and sleep state as design variables.** In deep sleep, a round is expensive because of the wake-up; in standby it is cheap. Choose E with the sleep model of the deployment, using Eqs. (4) and (7) and Algorithm 2.
4) **Count protocol overhead, not payload.** The payload takes under a millisecond of airtime, yet an exchange takes tens of milliseconds. Compression and quantization, including quantization-aware training [20], reduce the bits per update; on a link like ours, fewer connections or a persistent connection attack the larger cost.
5) **Watch for stalls.** A few exchanges take hundreds of milliseconds or more, add energy and raise the mean, so time-outs and retry rules matter for the energy budget.
6) **Leave a gap after a radio burst when measuring compute.** Without it, about 0.5 mJ of tail energy falls into the training window. In a deployed system, count that tail as communication.
7) **Expect contention to favor more local work.** If contention stretches each exchange by κ_c, the crossover grows about in proportion. This remains a hypothesis.
8) **Measure with an independent instrument and hardware markers.** Datasheet currents and analytical models miss the awake baseline, the protocol overhead and the tail (Section IV), and the integration rule for windows near the baseline changes the result (Appendix A). Algorithm 1 gives a procedure that keeps negative terms and rejects, rather than repairs, defective traces.

Other radios fit the same framework by replacing ΔE_{comm} and T_{comm}. A slow low-power wide-area link would push $E^\star$ much higher, and a short-range burst radio would pull it lower. This agrees with the split in the literature [24], [26], but we did not measure either radio.

X. Limitations

- We used one device, one board, one radio, one task and one trainable head, and the K clients are virtual.
- Communication energy varies considerably, with a standard deviation of 2 to 7 mJ per regime. The main campaign and the second session differ by 2.6 mJ at the same

setting for reasons we cannot identify, so the crossover is a range of about 7.5 to 10 epochs.
- The access point is a phone hotspot, and we did not log retries, channel load or the hotspot state.
- The tail test covers $E = 1$, 5 and 20 with five runs per cell, its main result has $p \approx 0.04$, and the extra energy at 0 ms is not constant across E. Its run order was blocked rather than randomized, and within each block the 0 ms cell always came first for a given E.
- Sleep power, wake-up energy, contention and payload size were not measured, so their results are model-only.
- Convergence is simulated on embeddings from the frozen backbone, for one task without the silence class and one level of label skew. We did not check bit-level agreement between the device and the simulation, and the energy-to-target table combines simulated rounds with measured per-turn energy.
- The analytical-model parameters come from ranges in the cited works; other choices change the errors, but not their direction.

XI. Conclusion

On an ESP32-S3 over Wi-Fi, measured with an independent power monitor, computation costs 1.00 mJ per epoch with no detectable constant term, and communication costs 9.7 mJ per client turn in a 60-run campaign and 7.4 mJ in a second session. The marginal crossover is therefore about 7.5 to 10 epochs, and 9.7 in the larger campaign. A radio burst leaves a tail of about 0.6 to 0.8 mJ, part of which is absorbed by training that starts immediately after the burst unless a short pause intervenes. Common analytical models miss the energies by 95% to 99% or give a crossover that is far too high or too uncertain, and the datasheet model is within about 10% only at its nominal point. When the device sleeps between rounds, the crossover moves toward the ratio of exchange time to epoch time, which with assumed sleep powers gives 11 to 19 epochs. In our convergence study, the energy to reach the target is smallest at $E = 10$ under every accounting. Sleep changes how costly it is to deviate from this value: $E = 20$ costs 44% more marginal energy but only 12% more with deep sleep, so the sleep regime is a power-management choice that shapes the choice of E. Measuring sleep power, wake-up energy and channel contention directly would turn the model-based parts of this paper into measured results.

Appendix A
Instrumentation Details

Sensing. An INA238 with a 15 mΩ (1%) shunt sits on the high side of the device supply. The ADC range is ±163.84 mV, the conversion time is 540 µs, and there is no averaging. A second ESP32-S3 reads current and bus voltage at 1 kHz and sends them over a USB serial link at 921,600 baud. Power is $P = V_{\mathrm{bus}} I$, and energy is a sum over 1 ms samples between marker edges.

Markers. GPIO 10 marks training and GPIO 11 marks communication. A 3-bit phase bus on GPIO 12, 13 and 14 labels sub-phases and is read by the logger. Each line has a 100 kΩ pull-down resistor.

Isolation. USB power is removed from the device during measurement, because a connected host would back-power the board and bypass the shunt. The device, the INA238 and the logger share one ground.

Supply. The bench supply gives 5.0 V DC to the board input, which feeds the on-board 3.3 V regulator. The multimeter read 5.04 V before and after the campaign, and the logged bus voltage averaged 5.03 V, with dips during radio and training bursts.

Quality gates and rejected runs. A run passes only if it has zero dropped samples, strictly increasing timestamps, the expected marker count and a baseline drift below 2%. The 105 accepted runs had a mean drift of 0.24% and a largest drift of 0.84%. Twenty attempts were rejected and repeated, and their traces are kept. Nineteen of them showed no training marker, which the bench log attributes to a serial reset when the port opened or to the phone hotspot going to sleep; we did not test these causes. One attempt had 99 dropped samples and an invalid first timestamp. The accepted traces were not edited.

Integration near the baseline. Energy is the sum of $(P - P_{\mathrm{base}}) \cdot 1\,\mathrm{ms}$ over a window, with negative terms kept. Clamping negative terms to zero would rectify baseline noise and add a positive bias that grows with the window length; on idle samples held out from the baseline estimate, this bias is 2.8 mW on average. It affects the communication windows, which are close to the baseline and long when the exchange stalls, but not the training windows, whose power is always above baseline. With clamping, the mean communication energy of the main campaign would be 10.16 instead of 9.75 mJ and the crossover 10.15 instead of 9.74 epochs. An independent implementation of Algorithm 1 accepts all 105 runs and reproduces their baselines to within 0.005 mW and their training and configuration energies to within 0.002 mJ. The reported communication energies take the window edges from the phase bus; in three runs these differ from the GPIO edges by one or two samples, which changes the main-campaign mean by 0.014 mJ and $E^\star$ by 0.01 epochs.

Appendix B
Status of Results

- *Measured:* P_{base}, ΔE_{comp}, ΔE_{comm}, the phase times and energies, and the tail profile, from 105 runs under one protocol.
- *Derived from measurements:* α, c_0, $E^\star$, the intervals, and the marginal, gross and deep-sleep energies in Table VII.
- *Assumed:* the 0.506 mJ tail charge in Version B (from the pilot campaign), the sleep power and wake-up energy, κ_c, and the analytical-model parameters.
- *Simulated:* the rounds $R(E)$ in Table VII.

Data and Code Availability

The 105 raw power traces, the run summaries, the analysis and figure scripts and the firmware of both boards are available at https://github.com/shakursiam/fl-energy-crossover.

REFERENCES

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, vol. 54, 2017, pp. 1273–1282.

[2] G. J. Pottie and W. J. Kaiser, "Wireless integrated network sensors," *Communications of the ACM*, vol. 43, no. 5, pp. 51–58, 2000.

[3] S. Wang, T. Tuor, T. Salonidis, K. K. Leung, C. Makaya, T. He, and K. Chan, "Adaptive federated learning in resource constrained edge computing systems," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 6, pp. 1205–1221, 2019.

[4] B. Luo, X. Li, S. Wang, J. Huang, and L. Tassiulas, "Cost-effective federated learning design," in *Proceedings of the IEEE Conference on Computer Communications (INFOCOM)*, 2021.

[5] N. H. Tran, W. Bao, A. Zomaya, M. N. H. Nguyen, and C. S. Hong, "Federated learning over wireless networks: Optimization model design and analysis," in *Proceedings of the IEEE Conference on Computer Communications (INFOCOM)*, 2019, pp. 1387–1395.

[6] Z. Yang, M. Chen, W. Saad, C. S. Hong, and M. Shikh-Bahaei, "Energy efficient federated learning over wireless communication networks," *IEEE Transactions on Wireless Communications*, 2021.

[7] M. Chen, Z. Yang, W. Saad, C. Yin, H. V. Poor, and S. Cui, "A joint learning and communications framework for federated learning over wireless networks," *IEEE Transactions on Wireless Communications*, vol. 20, no. 1, pp. 269–283, 2021.

[8] X. Zhou, J. Zhao, H. Han, and C. Guet, "Joint optimization of energy consumption and completion time in federated learning," in *2022 IEEE 42nd International Conference on Distributed Computing Systems (ICDCS)*, 2022, pp. 1005–1015.

[9] H. Chen, S. Huang, D. Zhang, M. Xiao, M. Skoglund, and H. V. Poor, "Federated learning over wireless IoT networks with optimized communication and resources," *IEEE Internet of Things Journal*, vol. 9, no. 17, pp. 16 592–16 605, 2022.

[10] T. Zhao, X. Chen, Q. Sun, and J. Zhang, "Energy-efficient federated learning over cell-free IoT networks: Modeling and optimization," *IEEE Internet of Things Journal*, vol. 10, no. 19, pp. 17 436–17 449, 2023.

[11] Q.-V. Pham, M. Le, T. Huynh-The, Z. Han, and W.-J. Hwang, "Energy-efficient federated learning over UAV-enabled wireless powered communications," *IEEE Transactions on Vehicular Technology*, vol. 71, no. 5, pp. 4977–4990, 2022.

[12] J. Ren, W. Ni, and H. Tian, "Toward communication-learning trade-off for federated learning at the network edge," *IEEE Communications Letters*, vol. 26, no. 8, pp. 1858–1862, 2022.

[13] X. Fan, Y. Chen, M. Liu, S. Sun, Z. Liu, K. Xu, and Z. Li, "UAV-enabled federated learning in dynamic environments: Efficiency and security trade-off," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 5, pp. 6993–7006, 2024.

[14] Z. Ju, F. Shen, and T. Wei, "BEFL: Balancing energy consumption in federated learning for mobile edge IoT," *arXiv preprint arXiv:2412.03950*, 2024.

[15] M. Hamood, A. Albaseer, H. El-Sallabi, M. Abdallah, A. Al-Fuqaha, and B. Hamdaoui, "Personalized federated learning with adaptive transformer pruning and hypernetwork-driven personalization in wireless networks," *IEEE Transactions on Machine Learning in Communications and Networking*, vol. 4, pp. 1–16, 2026.

[16] Z. Pan, Y. Li, Z. Guan, M. Liang, A. Li, J. Wang, and F. Kou, "RFCSC: Communication efficient reinforcement federated learning with dynamic client selection and adaptive gradient compression," *Neurocomputing*, vol. 612, p. 128672, 2025.

[17] J. Xiao, Y. Chen, C. Yang, K. Li, K. Xie, X. Li, K. Li, and W. Liang, "Communication-efficient federated learning based on a two-stage adaptive basis matrix for gradient compression," *IEEE Transactions on Mobile Computing*, vol. 25, no. 10, pp. 18 403–18 420, 2026.

[18] Y. Wang, L. Lin, and J. Chen, "Communication-efficient adaptive federated learning," in *Proceedings of the 39th International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 162, 2022, pp. 6292–6320.

[19] G. Paragliola, "Evaluation of the trade-off between performance and communication costs in federated learning scenario," *Future Generation Computer Systems*, vol. 136, pp. 282–293, 2022.

[20] X. Fang, L. Chen, H. Yin, X. Chen, and W. Wang, "Communication efficient federated learning with quantization-aware training design," *IEEE Transactions on Machine Learning in Communications and Networking*, vol. 4, pp. 45–59, 2026.

[21] N. Tekin, A. Acar, A. Aris, A. S. Uluagac, and V. C. Gungor, "Energy consumption of on-device machine learning models for IoT intrusion detection," *Internet of Things*, vol. 21, p. 100670, 2023.

[22] A. Sabovic, J. Fontaine, E. De Poorter, and J. Famaey, "Energy-aware tinyml model selection on zero energy devices," *Internet of Things*, vol. 30, p. 101488, 2025.

[23] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, C. Gan, and S. Han, "On-device training under 256KB memory," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[24] M. Ficco, A. Guerriero, E. Milite, F. Palmieri, R. Pietrantuono, and S. Russo, "Federated learning for IoT devices: Enhancing TinyML with on-board training," *Information Fusion*, vol. 104, p. 102189, 2024.

[25] S. Rupaak, G. Khekare, Y. Kumar, and G. Soni, "Quantifying energy and accuracy trade-offs of federated learning on wearable health devices," *Frontiers in Big Data*, vol. 9, p. 1769948, 2026.

[26] N. Llisterri Giménez, J. Miquel Solé, and F. Freitag, "Embedded federated learning over a LoRa mesh network," *Pervasive and Mobile Computing*, vol. 93, p. 101819, 2023.

[27] O. T. Sanchez, G. Borges, D. Raposo, A. Rodrigues, F. Boavida, and J. S. Silva, "Enhancing the performance of industrial IoT LoRaWAN-enabled federated learning frameworks: A case study," *Internet of Things*, vol. 32, p. 101632, 2025.

[28] A. Singh and S. S. Borkotoky, "Federated learning over LoRa networks: Simulator design and performance evaluation," *arXiv preprint arXiv:2508.10574*, 2025.

[29] H. Ren, D. Anicic, and T. A. Runkler, "TinyOL: TinyML with online-learning on microcontrollers," in *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, 2021.

[30] K. Kopparapu and E. Lin, "TinyFedTL: Federated transfer learning on tiny devices," *arXiv preprint arXiv:2110.01107*, 2021.

[31] N. Llisterri Giménez, M. Monfort Grau, R. Pueyo Centelles, and F. Freitag, "On-device training of machine learning models on microcontrollers with federated learning," *Electronics*, vol. 11, no. 4, p. 573, 2022.

[32] Y. Zhang, N. Suda, L. Lai, and V. Chandra, "Hello edge: Keyword spotting on microcontrollers," *arXiv preprint arXiv:1711.07128*, 2018.

[33] C. Banbury, V. J. Reddi, P. Torelli, N. Jeffries, C. Kiraly, J. Holleman *et al.*, "MLPerf tiny benchmark," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[34] N. Balasubramanian, A. Balasubramanian, and A. Venkataramani, "Energy consumption in mobile phones: A measurement study and implications for network applications," in *Proceedings of the 9th ACM SIGCOMM Internet Measurement Conference (IMC)*, 2009, pp. 280–293.

[35] X. Qiu, T. Parcollet, J. Fernandez-Marques, P. P. B. Gusmao, Y. Gao, D. J. Beutel, T. Topal, A. Mathur, and N. D. Lane, "A first look into the carbon footprint of federated learning," *Journal of Machine Learning Research*, vol. 24, pp. 1–23, 2023.

[36] *ESP32-S3 Series Datasheet*, Espressif Systems, 2024, v1.6.

[37] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," *arXiv preprint arXiv:1804.03209*, 2018.